

Documentación de Cierre: Módulo 2 – El Motor de Simulación y Control de Tiempo

1. ¿Qué hace este módulo?

El Módulo 2 es el "corazón" de **SynthCity**. Lo que hace es coger la ciudad del Módulo 1 (que era un mapa estático con bloques puestos ahí sin más) y meterle vida. Para ello, ejecuta una simulación por **ciclos** (turnos).

En cada turno, el motor calcula cómo interactúan los bloques: cuánta energía se fabrica y se gasta, si los servicios públicos cubren la demanda de los ciudadanos, si el transporte es eficiente y cómo afecta la contaminación a la felicidad (bienestar) y a la estabilidad de la ciudad.

2. Cómo movemos los datos (ResultadoSimulacion y MetricaCiudad)

Para que los datos no se corrompan y todo quede bien ordenado, hemos creado dos clases principales que guardan la información "con candado" (son inmutables, no se pueden modificar una vez creadas):

ResultadoSimulacion (Módulo 2)

Es el informe final que escupe el simulador cuando termina. Guarda:

- Datos básicos: nombre de la ciudad, tamaño de la cuadrícula (\$20 \times 20\$, etc.) y cuántos bloques hay activos o inactivos.
- El historial completo: una lista con los datos exactos de lo que pasó en cada ciclo.
- Métricas del final: la contaminación total que se acumuló, el bienestar más alto, el más bajo y la tendencia (si la ciudad iba a mejor o a peor al final de la simulación).

MetricaCiudad (Módulo 3)

Esta clase agarra el informe anterior y lo procesa para que el módulo de analítica lo entienda. Aquí dentro guardamos los **umbrales del juego**:

- UMBRAL_DENSIDAD_RIESGO = 0.85: Si más del 85% de la ciudad está llena, la cosa empieza a saturarse.
- UMBRAL_CONTAMINACION_ALTA = 60.0: A partir de 60 de contaminación, el aire ya es tóxico.
- UMBRAL_RATIO_DEFICIT = 0.80: Si la energía o los servicios caen por debajo del 80% de lo que se necesita, entramos en déficit.

3. Las Fórmulas del Proyecto (Índices Complejos)

Para saber si una ciudad va bien o mal, usamos tres fórmulas que calculan notas de 0.0 a 1.0 (gracias a un método clamp01 que inventamos para que ningún valor se salga de ese rango).

I. Índice de Equilibrio Base

Mide cómo de compensada está la ciudad (que no haya solo casas o solo industrias).

- **Fórmula:** Un 40% depende de los bloques activos, un 20% de la diversidad de tipos de bloques, un 10% del ratio de energía, un 10% del de servicios y un 10% del transporte.
- **Bonus:** Si la presión industrial es bajita (≤ 10) sumamos $+0.05$. Si la densidad no es agobiante (≤ 0.80) sumamos otros $+0.05$.

II. Índice de Viabilidad Base

Es la nota que dice si la ciudad es sostenible para el futuro.

- **Fórmula:** Un 30% los bloques activos, un 20% el ratio de energía, un 20% el ratio de servicios, un 15% la estabilidad y un 15% el bienestar.
- **Penalizaciones:** Le restamos un 10% de la contaminación dividida entre 100, y otro 10% si la densidad supera el límite de riesgo (0.85).

III. Índice de Saturación

Mide cómo de "ahogada" está la ciudad. Empezamos con el valor de la densidad actual y le **sumamos 0.05 puntos obligatorios** por cada problema que encontremos:

- ¿Hay apagones o falta energía? $+0.05$.
- ¿Faltan hospitales, colegios o servicios? $+0.05$.
- ¿La contaminación supera el umbral de 60? $+0.05$.

4. Evitando errores: Validaciones a prueba de bombas

Para asegurarnos de que nadie meta datos erróneos o "haga trampas" sin querer en el código, los constructores de las clases tienen un control de errores súper estricto. Si algo falla, lanzamos un `ResultadoSimulacionInvalidoException` (nuestra propia excepción personalizada).

Controlamos cosas como:

- Si sumas los bloques activos y los inactivos, **tiene que dar** el total de bloques por cojones.
- Ningún contador de energía, bloques, ciclos o contaminación puede ser negativo (≤ 0).
- El balance de energía tiene que cuadrar exactamente restando la producida menos la consumida.

- Los porcentajes y notas (bienestar, estabilidad, etc.) tienen que estar a la fuerza entre 0.0 y 1.0.
- En el historial, los ciclos tienen que ir seguidos (\$1, 2, 3...\$) y la contaminación acumulada siempre tiene que ir hacia arriba, nunca bajar.

5. Pruebas Unitarias (JUnit 5)

Para demostrar que el motor funciona de diez, hemos programado dos tests automatizados utilizando una ciudad de prueba con 3 casas, 1 zona de servicios, 1 estación de transporte y 2 industrias:

Test de Duración (testCiudadPruebaTieneVariosCiclos)

Comprueba que la ciudad de prueba está lo bastante equilibrada como para durar **al menos 2 ciclos seguidos** simulando, sin romperse o vaciarse en el primer turno.

Test de Determinismo (determinismo)

Este es el más importante. El simulador **no puede ser aleatorio**. Si metes la misma ciudad dos veces seguidas, el resultado tiene que ser exactamente idéntico.

El test simula dos veces la misma ciudad y comprueba con un assertEquals (con un margen de error ridículo de 10^{-9} para los decimales) que coinciden:

- Los ciclos totales y el motivo por el que se paró la simulación.
- Las tendencias de contaminación y felicidad.
- **Paso por paso:** Se mete dentro de la lista de ciclos y revisa, uno a uno, que el ciclo 1 de la primera simulación tenga los mismos kilovatios, los mismos datos de transporte, la misma contaminación y el mismo bienestar que el ciclo 1 de la segunda simulación. Y lo mismo con el ciclo 2, 3, etc.

6. Conclusión

El Módulo 2 ha quedado fino y funciona perfectamente. Consigue aislar el mapa del juego de las movidas matemáticas de los ciclos. Al usar colecciones inmutables (como Map.copyOf o List.copyOf), nos aseguramos de que nadie pueda modificar los datos de una simulación ya terminada, y los tests de JUnit nos garantizan que el motor es 100% fiable para cuando tengamos que meter la expansión automática en el Módulo 3.